

Práctica 6:

Control Con Arduino

Temas tratados:

- Control digital.
- Conversor Analógico-Digital.
- Salidas PWM.
- Control Todo-Nada.

Material utilizado:

- Bastidor de dos niveles.
- Fuente de alimentación ± 15 V (72686)
- Generador de la variable de referencia (73413)
- Sistema Térmico (elegir una opción):
 - Amplificador de potencia (72603) y Módulo térmico (73412)
 - Módulo térmico (734121)
- Arduino con material electrónico
- Placa Protoboard.
- Módulo Arduino

6.1. Antecedentes

En este apartado se hará una pequeña introducción al control digital y a la implementación de un algoritmo de control en un microcontrolador, en este caso un Arduino. Para ello, se hará un breve repaso a conceptos como periodo de muestreo, resolución de entradas analógicas, salidas PWM, etc.

6.1.1.- Nociones de control digital

Hasta ahora hemos analizado sistemas analógicos, en los que la variable de control se realimentaba y de forma “inmediata” se producía la acción de control. Es decir, sistemas en los que casi no existe retardo dentro del controlador, hay que tener en cuenta que los sistemas con retardo nulo no existen, pero si el retardo es muy inferior a las constantes de tiempo del sistema, estos retardos se pueden despreciar.

Que ocurre cuando queremos controlar un sistema con un dispositivo informático: microcontrolador, ordenador, Raspberry, ... A grandes rasgos, estos dispositivos tienen que leer la variable de control y la referencia, con esta información deben de aplicarle el algoritmo de control y finalmente deben de sacar por su salida analógica la acción de control a aplicar al sistema. Esto implica retardos dentro del dispositivo de control, que dependiendo del proceso a controlar puede ser que deban ser tenidos en cuenta.

Veamos que ocurre en una escala de tiempos cuando se ejecuta un ciclo de control y posteriormente los desarrollaremos en más profundidad:

1. Se genera la variable de control o de proceso en el transductor. En el caso del sistema de prácticas, el transductor lee la temperatura y la convierte en una señal de tensión con un rango de 0-10V, equivalentes a un rango de temperaturas de 0-100°C. Aquí aparece la primera constante de tiempo del sistema, ya que hay un cierto retardo entra la temperatura medida y la tensión en el transductor.
2. La tensión generada por el transductor, hay que pasarla a valores seguros para el Arduino. En nuestro caso, el transductor genera una tensión entre 0 y 10V, pero la entrada del Arduino soporta como máximo señales de 0 a 5 V, por tanto, hay que poner un divisor de tensión para realizar esta tarea. Esta tarea es instantánea y no aporta retardo.
3. La señal en tensión conectada a una entrada analógica del dispositivo de control, hay que pasarla a un valor digital. Esto lo realiza el conversor analógico-digital. En el Arduino Uno, el conversor analógico digital es de 10 bits, por lo que una entrada analógica de 0-5V, se pasa a un número de $0 - (2^{10} - 1)$, es decir, de 0-1023. Este proceso provoca un retardo, que en el caso del Arduino Uno es menor de 1ms. Este valor debe ser tenido en cuenta para controles muy rápidos.
4. a) Ya en el programa de control la señal de entrada habrá que escalarla al valor de la variable de proceso. Igual ocurre si la consigna del sistema también entra a través de una entrada analógica. Este proceso también es del orden de microsegundos y no hay que tenerlo en cuenta salvo para controles muy rápidos.

- b) Una vez que se disponga de las señales anteriores, se ejecutará el algoritmo de control y se generará la salida de control. Este proceso depende de la complejidad del algoritmo, pero en el caso de nuestra práctica es del orden de microsegundos. PERO, lo que se hace en los sistemas de control es que el algoritmo de control se ejecute a periodos de tiempo predefinidos, es lo que se llama periodo de muestreo. Es decir, que el algoritmo de control se ejecuta y no vuelve a ejecutarse hasta que empiece el siguiente periodo de muestreo. Esto implica que dentro del programa deberá haber una especie de reloj que avisa cuando hay que ejecutar el algoritmo.
5. La salida generada por el programa se envía a la salida analógica o a una salida PWM:
- Salida analógica (no disponible en el Arduino Uno): la salida generada por el programa se envía a un conversor digital-analógico, de manera que en las bornas de salida del micro aparecerá una tensión proporcional al valor de la variable. Por ejemplo, si el conversor digital analógico es de 8 bits el Arduino convertirá una variable de $0 - (2^8 - 1)$, es decir, de 0-255 a una tensión en bornes de 0-5V.
 - Salida PWM (disponibles en el Arduino Uno): se convertirá una señal de 0-255 (si el conversor es de 8 bits) a una modulación PWM de 0 al 100%.
- En ambos casos este proceso es inferior a 1 ms.
6. Debido a que la entrada del sistema térmico es de 0-10V y la salida del Arduino es de 0-5V, habrá que duplicar la salida del Arduino con un amplificador operacional y su circuito correspondiente. Este proceso también es del orden de uno pocos microsegundos.
7. Finalmente se aplica la acción de control al sistema térmico y éste cambia su temperatura que se mide como en el paso 1. Se repite el ciclo.

Aquí se muestran varios temas a tener en cuenta en los sistemas de control digital:

1.- Resolución de los convertidores analógico-digitales: Estos son los encargados de convertir la tensión en voltios en la entrada analógica correspondiente en un valor numérico en el programa de control. Por ejemplo, en el sistema térmico, el transductor convierte $(0 - 100^{\circ}\text{C}) \rightarrow (0 - 10\text{V})$, el divisor $(0 - 10\text{V}) \rightarrow (0 - 5\text{V})$ el conversor analógico digital del Arduino Uno de 10 bits $(0 - 5\text{V}) \rightarrow (0 - 1023)$ y el factor de conversión en el programa de control debe hacer que se mida la variable real, por tanto $(0 - 1023) \rightarrow (0 - 100^{\circ}\text{C})$. Por tanto, se observa que la máxima precisión en la medida, es decir, a cuanto equivale una unidad de salida del conversor analógico digital, viene dada por,

$$\text{Resolución} = \frac{100 - 0}{1023 - 0} = 0.098 \approx 0.1^{\circ}\text{C} \quad (1)$$

Es decir que cada vez que la salida del conversor analógico digital aumenta en una unidad, aumentará la salida medida en 0.1°C . Por tanto, con este conversor analógico digital no se podrán medir incrementos de temperatura inferiores a 0.1°C . Si hiciera falta medir incrementos de temperatura menores, entonces habría que elegir un conversor analógico digital externo de un mayor número de bits.

2.- Resolución de los convertidores digital-analógico o PWM: Al igual que en el caso anterior, marca la resolución de las salidas del controlador.

3.- Tiempos de conversión y de ejecución de programas: Estos son los tiempos que tarda el sistema de control en las conversiones y en ejecutar los algoritmos. Sólo tiene que ser tenido en cuenta, en el caso que nos ocupa, si el sistema a controlar tiene una constante de menos de 10 ms, en el caso de nuestro sistema es de 60s.

4.- Periodo de muestreo: Los sistemas de control digital suelen trabajar con periodos de tiempo constante. Es decir, el bucle lectura de datos, algoritmo de control y salida de acción de control se realiza a intervalos de tiempo constantes. Este intervalo de tiempo se selecciona a la hora de calcular el sistema de control y generalmente todos los algoritmos de control dependen de este valor, por lo que si se cambia este intervalo de tiempo, probablemente haya que cambiar algunos parámetros en los algoritmos de control.

Para entender su significado, vamos a suponer que sólo queremos tomar los datos de un sistema para mostrarlos por pantalla. Supongamos que queramos mostrar por pantalla la salida de nuestro sistema térmico en lazo abierto. Este sistema puede tener una constante de tiempo de 60 segundos y el tiempo en llegar al 95% de su valor final ante una entrada en escalón está en torno a los 180s. La figura 6.1 muestra la toma de datos en función de los distintos periodos de muestreo. Veamos que ocurre en cada uno de los casos en función de los diferentes periodos de muestreo T .

$T=0,1s$ y $T=1s$: Se observa que todas las posibles variaciones de temperatura del sistema, aún las más rápidas serán captadas.

$T=3s$: Se observa que se pierde algo de información, pero que los datos tomados permiten reconstruir la curva con mucha precisión.

$T=10s$: Se observa que se pierde algo de información, pero se puede reconstruir la señal

$T=30s$: Se pierde una gran cantidad de la información del sistema. Hay que tener en cuenta que si utilizamos este tiempo de muestreo para el control, desde que se detecta un valor de temperatura hasta que se actúe, pasan 30 segundos, lo que se observa claramente que penalizaría el control.

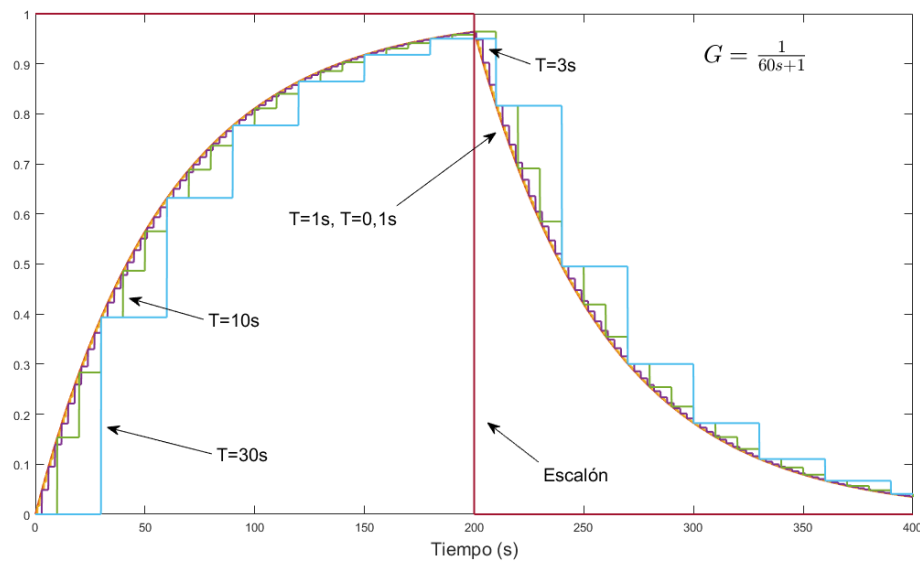


Figura 6.1.- Efectos del periodo de muestreo.

Se observa que el periodo de muestreo está muy relacionado con la señal que se desee capturar o controlar. Es decir, es muy importante saber la constante de tiempo o la frecuencia del sistema a la hora de elegir un periodo de muestreo adecuado. En el caso que nos ocupa se observa que con un periodo de muestreo de 1 segundo o menos no se pierde información del sistema, con un periodo de muestreo 20 veces inferior a la constante de tiempo $T=3s$ la curva se puede reconstruir con mucha precisión. Se observa que si el periodo de muestreo es muy elevado, no se podrá regenerar la señal original. Para el caso que nos ocupa vamos a tomar un periodo de muestreo de 1s.

6.1.2.- Nociones de Arduino Uno

Este apartado no pretende ser una introducción al Arduino Uno. Sólo se expondrán los aspectos que se consideren más importantes para la realización de la práctica. Para más información existen una gran cantidad de foros en los que se pueden resolver todas las dudas que pudieran surgir.

El Arduino Uno tiene las siguientes características:

Basada en el microcontrolador Microchip ATmega328P6.

En la práctica se alimentará a través del puerto USB, aunque permite una alimentación externa con una tensión de entre 7 y 20V.

Su tensión de funcionamiento es de 5V, por lo que no se debe someter a ninguna entrada ni salida a una tensión superior a 5V.

Dispone de 14 pines de entrada salida de los que 6 son PWM.

Dispone de 6 pines de entradas analógicas de 10 bits.

Todas las tensiones de la placa están referidas a sus pines GND.

La placa genera tensiones de 3,3V y de 5V de poca potencia.

6.1.3.- Nociones del IDE de Arduino

El IDE de Arduino es el entorno de programación para los microcontroladores de la marca Arduino. Su aspecto se puede ver en la figura 6.2.

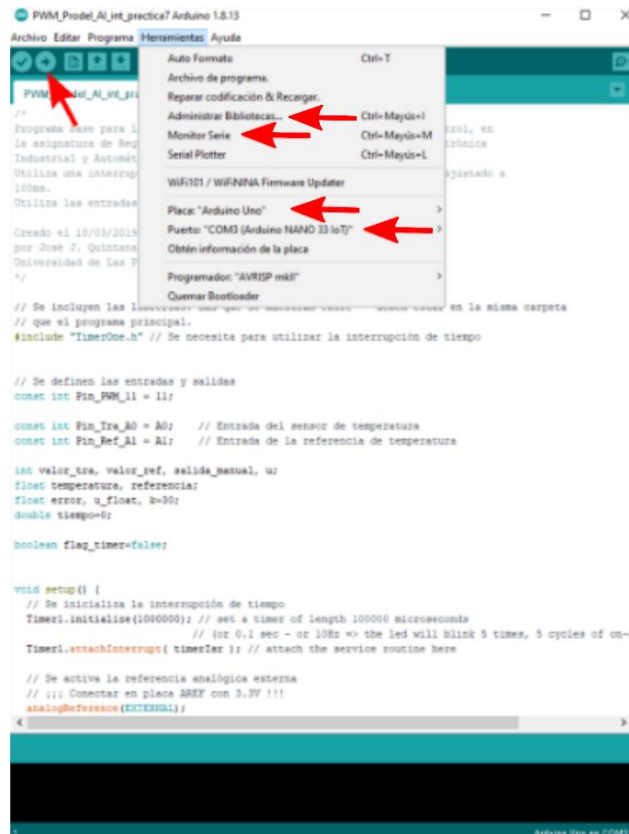


Figura 6.2.- IDE Arduino

Los ajustes que se van a utilizar son los marcados en rojo en la figura 6.2. Si no se detecta automáticamente se debe elegir la **placa** del menú desplegable, en nuestro caso será el **Arduino Uno**. A continuación, en el menú desplegable del **puerto**, se debe seleccionar el que pone **Arduino entre paréntesis**.

El botón que tiene una flecha se usa para compilar el programa y volcarlo en el Arduino.

El monitor serie se utiliza para ver los datos que se mandan por el puerto serie, en la práctica que se va a realizar, se manda: el tiempo, la consigna, la temperatura y la señal de control. Para la toma de datos, se seleccionarán los datos y se copiarán al portapapeles y a continuación en un fichero de texto.

6.1.4.- Estructura de un programa de Arduino

En el anexo 6.1 se puede observar el programa de Arduino que servirá como base para la realización de esta práctica. Se tomará este programa para explicar por encima, la estructura de un programa de Arduino:

- **El encabezado:** suele indicar como en todos los programas: el autor, la fecha, el título del programa y la versión. También en esta parte se hace un resumen de lo que hace el programa.
- **Declaraciones y carga de librerías:**
 - o Se cargan librerías que no se encuentran en el lenguaje estándar. Por ejemplo, en este código se carga la librería `#include "TimerOne.h"` que se utilizará para generar interrupciones relacionadas con el periodo de muestreo.
 - o Declaración de variables y constantes (en adelante se llamarán a todas variables): en este bloque se declaran las variables, se dice de que tipo son (esto no hace falta hacerlo en algunos lenguajes, pero aquí es obligatorio) y si se desea se puede también inicializar el valor.
 - o Declaración y asignación de las entradas y salidas físicas.
- **Bloque setup:** se inicializan parámetros relacionados con programa, en este código se inicializan, por ejemplo:
 - o El intervalo de tiempo en el que la interrupción de tiempo se activa. El comando es
`Timer1.initialize(1000000);`
y da el periodo en microsegundos. En este caso la función asociada a la interrupción se activa cada $1000000\mu s = 1s$. La función asociada a esta interrupción se define en el comando
`Timer1.attachInterrupt(timerIsr);`
que indica que cada vez que pase ese tiempo se dejará de correr el programa y se ejecutará la función
`timerIsr`
Para poder utilizar estos comandos es necesario haber instalado la librería
`#include "TimerOne.h"`.
 - o Se asigna una referencia de tensión analógica externa. Por defecto el rango de las entradas analógicas en el Arduino Uno es de 0-5V, pero para evitar problemas de sobretensiones se puede alterar este valor conectando en el pin Aref el valor de tensión para el nuevo rango. Como el Arduino dispone de una fuente de tensión de 3,3V, se deben conectar ambos pines tal como se muestra en el esquema. El comando
`analogReference(EXTERNAL);`
Hace que el rango de la entrada analógica sea a partir de ahora
 $(0 - 3,3V) \rightarrow (0 - 1023)$
Este rango se ha utilizado, ya que el divisor divide por 3 y la tensión máxima del equipo puede ser de 15V, pero en la operación normal la tensión máxima es de 10V, que se corresponde con 3,3V en la entrada analógica correspondiente.

- Se define el pin 11 como pin de salida
`pinMode(Pin_PWM_11, OUTPUT);`
- Se inicializa el puerto serie (USB conectado al PC) para ser utilizado en las comunicaciones con el programa, además de define la velocidad a 9600.
`Serial.begin(9600);`
- **Bloque loop:** en este bloque es donde está el código del programa. En este bloque cuando se llega al final de él, se vuelve a ejecutar desde el principio, es decir, que se estará ejecutando de forma cíclica mientras el Arduino esté en modo Run.
 - La primera línea se ejecuta cada vez que la variable (flag) `flag_timer=1`, esta variable **se pone a 1 cada vez que se ejecuta la función**
`timerIsr`
Esta función se activa cada segundo como se comentó anteriormente. Por otra parte, la variable `flag_timer`, **se pone a cero cuando se finaliza el bloque if**. Por tanto, sólo se ejecuta el código interno del
`if (flag_timer)`
cada vez que se activa la interrupción.
 - Se mide el tiempo de la prueba. Debido a que el tiempo de muestreo es de 1s, la variable `tiempo` mide el tiempo en segundos de la prueba, con el comando
`tiempo=tiempo+1;`
 - Lectura de las señales de entrada analógica mediante el comando
`valor_ref = analogRead(Pin_Ref_A1);`
 - Escalado de las variables analógicas que están un rango de (0 – 1023) al valor de la variable de proceso.
`referencia=(float)valor_ref*100/1023;`
Hay que hacer notar que
`valor_ref`
es una **variable entera** y que
`referencia`
es una variable **tipo float** (es decir que tiene decimales).
 - Enviar el valor analógico al pin salida. Para ello la variable de salida debe tener un rango de (0 – 255) salida PWM de 8 bits, esta conversión se realiza con el siguiente comando
`salida_manual=map(valor_ref,0,1203,0,255);`
A continuación se envía el valor al pin correspondiente con el comando
`analogWrite(Pin_PWM_11, salida_manual);`
 - En caso de que se vaya a realizar un control proporcional, se deben utilizar estas variables
`error, u_float, k;`
Y antes de enviar la salida `u_float` a la salida PWM, hay que pasarla a un valor entero con el comando
`u_int=(int)u_float;`
 - Envío de datos al puerto serie: Se hará con el comando `serial.print`. Si se quiere que en el monitor serie aparezca que es cada variable se debe poner (en este caso no se va a utilizar):
`//Serial.print("tiempo = "); // En caso de que se quiera mostrar por el puerto serie`
Se envía el valor del tiempo al puerto serie.


```
Serial.print(tiempo);
```

Se envía una tabulación para separar las variables que se envían.

```
Serial.print("\t "); // Tabulación
```

Se envía la siguiente variable.

```
//Serial.print("\t referencia = ");
```

Se termina la línea para enviar otro grupo de datos

```
Serial.print("\n"); // Nueva línea
```

Estos datos se muestran si se abre la pantalla del monitor serie.

- Por último se pone la variable `flag_timer` a `false`.

```
flag_timer=false;
```

Para indicar que se tiene que volver a activar la interrupción para volver a ejecutar el bucle de control.

- **Función `timerIsr`**: es la función asociada a la interrupción de tiempo. Cada vez que pasa 1 segundo exacto, se activa esta interrupción y se ejecuta lo que hay en su interior.

```
void timerIsr()
{
    flag_timer=true;
}
```

En este caso lo único que se hace es poner la variable `flag_timer` a `true`, para indicar al programa principal que se debe ejecutar el algoritmo de control.

6.1.5.- Implementación de un control PID en el Arduino

El código que se anexa en el anexo 6.1, muestra tres controles para los que no se necesita tener en cuenta la frecuencia de muestreo, ya que no necesitan de los estados anteriores ni de las variables de entrada ni de la salida. Tal como se ha comprobado en prácticas anteriores, la única manera de eliminar el error en régimen permanente a nuestro sistema térmico es poniéndole un integrador a nuestro sistema en la cadena directa, o sea un controlador PI o PID. En este apartado se describirá brevemente como realizarlo.

Tal como se ha visto en apartados anteriores, el controlador PID está formado por tres acciones distintas, lo que se hará a continuación es calcular cada una de ellas por separado y luego sumarlas. Para el desarrollo, la entrada al controlador será el error e y la salida y .

Acción proporcional: viene dada por la siguiente ecuación

$$y_p(k) = ke(k) \quad (6.2)$$

Es decir, la salida en cada periodo de muestreo será el error multiplicado por la ganancia proporcional del controlador k .

Acción derivativa: en el dominio de Laplace, la ecuación de esta acción viene dada por

$$y_d(s) = kT_d s \quad (6.3)$$

Si aplicamos la aproximación de la derivada hacia atrás para convertir el sistema del dominio S al Z , tenemos

$$G_d(z) = \frac{y_d(z)}{e(z)} = kT_d \frac{z-1}{Tz} = \frac{kT_d(1-z^{-1})}{T} \quad (6.4)$$

Operando para obtener la ecuación en z negativas,

$$Ty_d(z) = kT_d e(z) - kT_d z^{-1} e(z) \quad (6.5)$$

Se llega a la ecuación en diferencias para el control derivativo

$$y_d(k) = k \frac{T_d}{T} (e(k) - e(k-1)) \quad (6.6)$$

Acción integral: En el dominio de Laplace, la ecuación de esta acción viene dada por la siguiente ecuación,

$$y_I = \frac{k}{T_i s} \quad (6.7)$$

Donde k es la ganancia proporcional, si aplicamos la aproximación de Tustin para pasar del dominio S al dominio z, tenemos

$$s \approx \frac{2(z-1)}{T(z+1)} \quad (Tustin) \quad G_I(z) = \frac{y_I(z)}{e(z)} = \frac{kT(z+1)}{2T_i(z-1)} = \frac{kT(1+z^{-1})}{2T_i(1-z^{-1})} \quad (6.8)$$

Desarrollando la ecuación en z negativas tenemos,

$$y_I(z) - z^{-1}y_I(z) = \frac{kT}{2T_i} (e(z) + z^{-1}e(z)) \quad (6.9)$$

De la que se obtiene la ecuación en diferencias de la acción integral,

$$y_I(k) = y_I(k-1) + \frac{kT}{2T_i} (e(k) + e(k-1)) \quad (6.10)$$

Controlador PID: Debido a que la acción integral depende del valor anterior de la acción integral, para la realización de este controlador hay que ejecutar estas dos ecuaciones,

$$y_I(k) = y_I(k-1) + \frac{kT}{2T_i} (e(k) + e(k-1)) \quad (6.11)$$

$$y_{PID} = ke(k) + k \frac{T_d}{T} (e(k) - e(k-1)) + y_I(k)$$

IMPORTANTE: Hay que tener en cuenta que el **periodo de muestreo T** que se ponga en las **ecuaciones 6.11** debe **coincidir** con el periodo de muestreo que **hayamos programado en el Arduino**.

Se observa que la acción proporcional está representada en color rojo, la derivativa en color verde y la integral en color azul. Si se quiere eliminar la acción derivativa o integral, bastará con eliminar el código de su color correspondiente. Si se desea implementar en **esta práctica**, es recomendable **eliminar la acción derivativa** y controlar el sistema con un **control PI**.

En el diagrama de flujo siguiente se muestra como implementar este control en el Arduino u otro dispositivo programable.

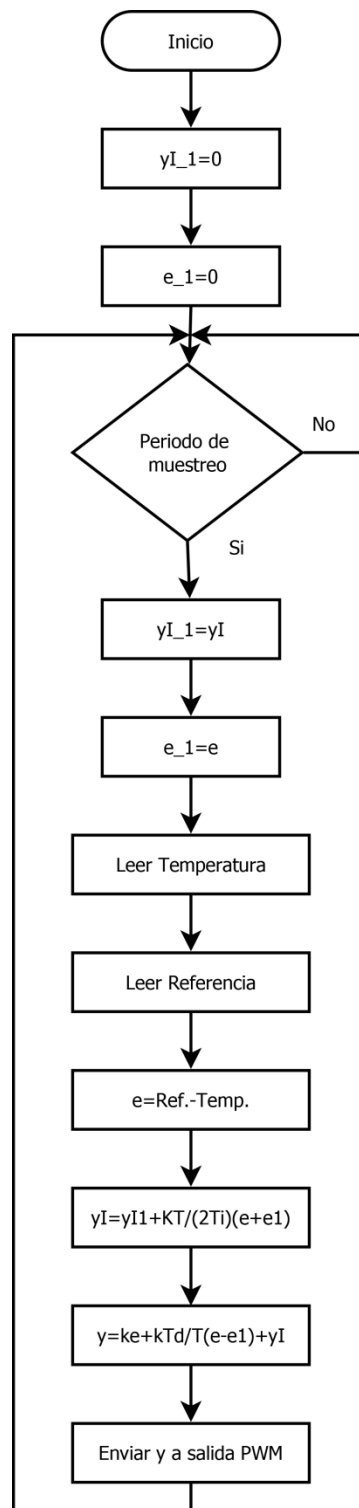


Figura 6.3.- Diagrama de flujo del controlador PID.

6.2 En el laboratorio

Esta práctica se puede realizar de dos maneras distintas, por una parte realizando los estudiantes el módulo de control mediante una protoboard y por otra utilizando el módulo Arduino. El montaje utilizando ambas opciones se detalla en los apartados 6.2.1 y 6.2.2.

6.2.1.- Montaje de la práctica (con protoboard)

Las figuras 6.4 y 6.5 muestran los puntos para conectar el módulo de prácticas a la placa de control de Arduino según el esquema de la figura 6.6.

Con el sistema **apagado** (interruptor $I_{on} = 0$), hemos de realizar las conexiones que se indican según corresponda.

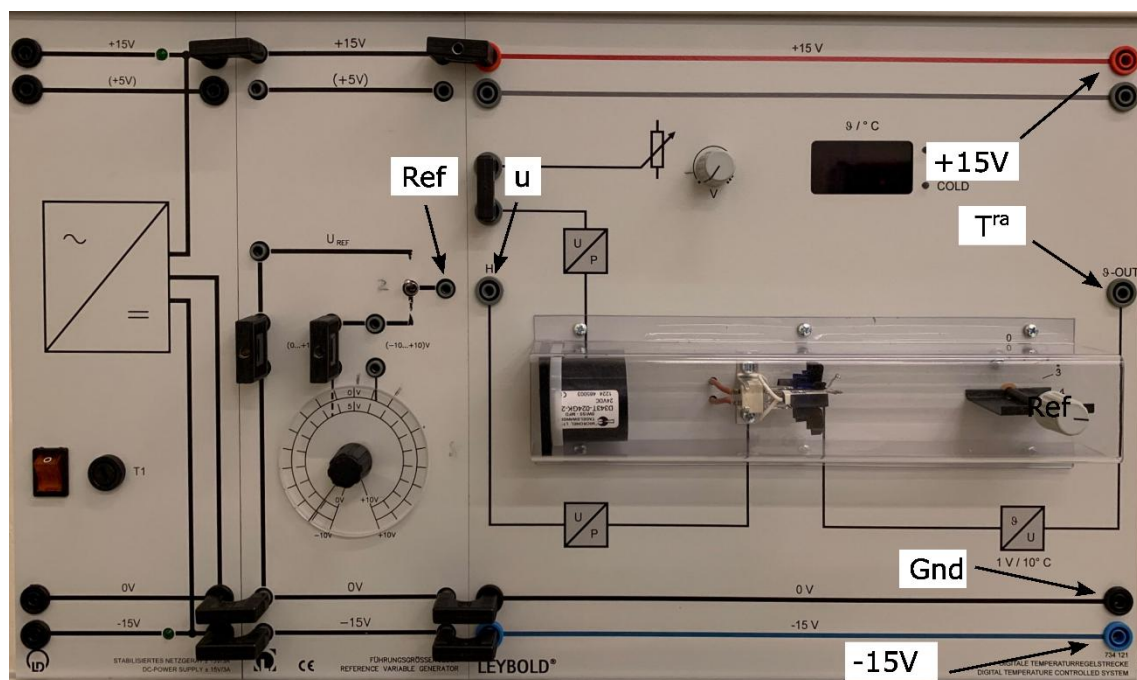


Figura 6.4.- Conexiones para el equipo nuevo

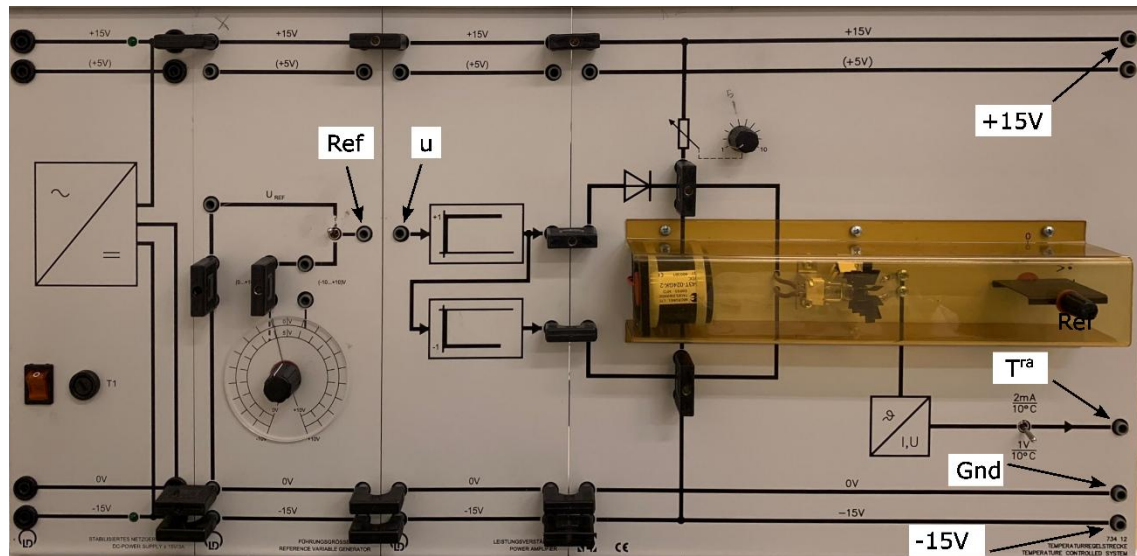


Figura 6.5.- Conexiones para el equipo antiguo

Las conexiones del Arduino a la placa protoboard y al equipo de prácticas se muestran en la figura 6.6.

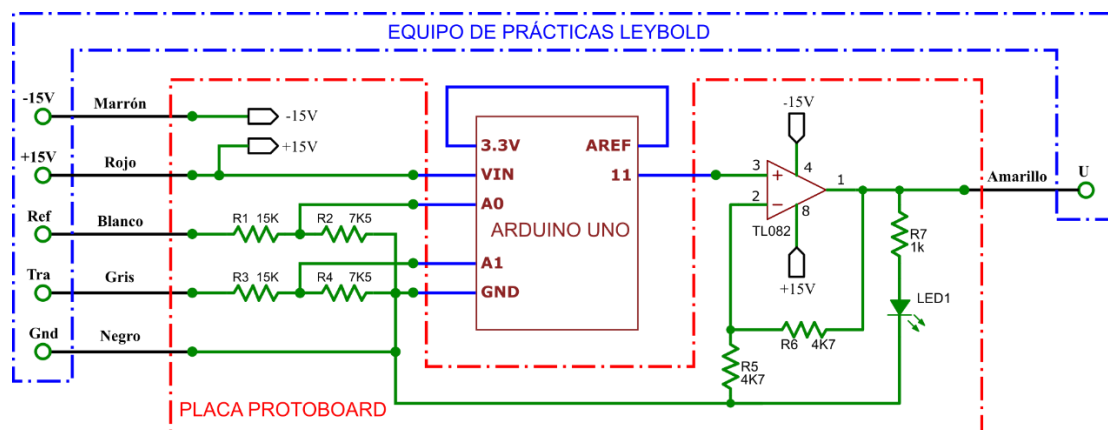


Figura 6.6.- Conexión equipo de prácticas – Protoboard – Arduino.

Las conexiones entre el equipo de prácticas y la placa protoboard se harán mediante un multicable marcado en negro en la **figura 6.6**. Las conexiones entre la protoboard y el Arduino se harán con los cables suministrados.

También se anexa el conexionado del TL082 en la figura 6.7, para conocer cómo se numeran los pines y cuál es la marca de referencia del circuito integrado.

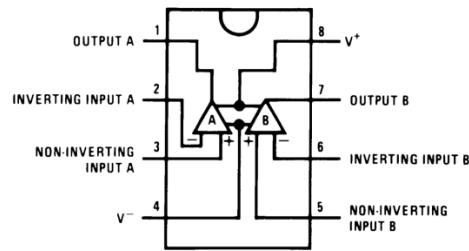


Figura 6.7.- Conexión interna del TL082.

En el Arduino es necesario conectar el pin **3.3V** al pin **Aref**, como se muestra también en la figura 6.6.

6.2.3.- Cosas a tener en cuenta y otras que pueden ir mal

1.- Si se han hecho bien las conexiones, el **negativo** del **polímetro** se puede conectar al **0V del equipo térmico**, para medir todas las tensiones.

MUY IMPORTANTE: Antes de **conectar** los cables que van a los **pines A0 y A1 del Arduino**.

- En el equipo térmico, poner el **potenciómetro de referencia al máximo (10V)** y con el cable **blanco conectado** y comprobar que a la **salida del divisor** hay **3.3V** (el que va a la entrada A0). Si todo está **bien conectar** el cable correspondiente en **A0**.
- **Desconectar** el **cable blanco de Ref** y **conectar** en su lugar el **cable gris** y **comprobar** de nuevo que al poner 10V en el cable gris a la **salida del divisor** de tensión de la **temperatura** (la que iría a la entrada A1) marca también **3,3V**. Si todo está **bien conectar** el cable correspondiente en **A1**.

Esta comprobación es muy importante, ya que, si el divisor se cableara mal y entrarán más de 5V a cualquiera de las entradas del Arduino, **la entrada se destruiría**.

2.- Una vez hecho el montaje y comprobado, no apagar el equipo térmico con su interruptor general. Ya que esto a veces hace que falle el puerto de comunicaciones con el Arduino.

3.- Si se **intenta cargar el programa** en el Arduino y **da problemas de comunicación** (teniendo bien seleccionado el puerto serie), lo más probable es que se haya bloqueado el puerto. **Hay que reiniciar el ordenador**.

4.- En la placa protoboard del laboratorio que se muestra en la figura 6.8, las **conexiones** de las **hilas de pines** marcados como + y – están marcados en **rojo** y hay que tener en cuenta que **están desconectados** en la **parte central** como se muestra en la figura.

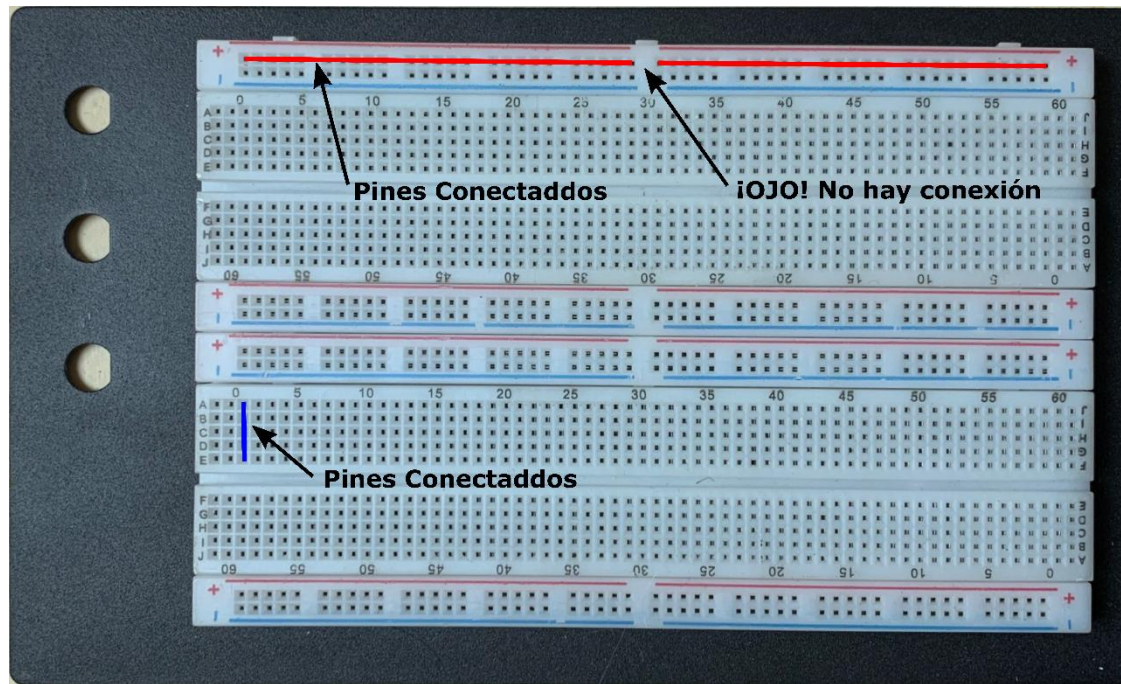


Figura 6.8.- Conexionado interno de la placa protoboard del laboratorio.

Por otra parte, las hileras de los pines marcados como **ABCDE** y **FGHIJ** están conectados como está marcado en azul.

6.2.2.- Montaje de la práctica (con módulo Arduino)

Las figuras 6.9 y 6.10 muestran las conexiones de los equipos de prácticas cuando se utiliza el módulo de control Arduino y el módulo térmico nuevo. La **figura 6.9** será la utilizada para el control utilizando el **IDE de Arduino**, mientras que la **figura 6.10** será la utilizada para el control utilizando el **Simulink**.

Con el sistema **apagado** (interruptor $I_{on} = 0$), hemos de realizar las conexiones que se indican según corresponda.

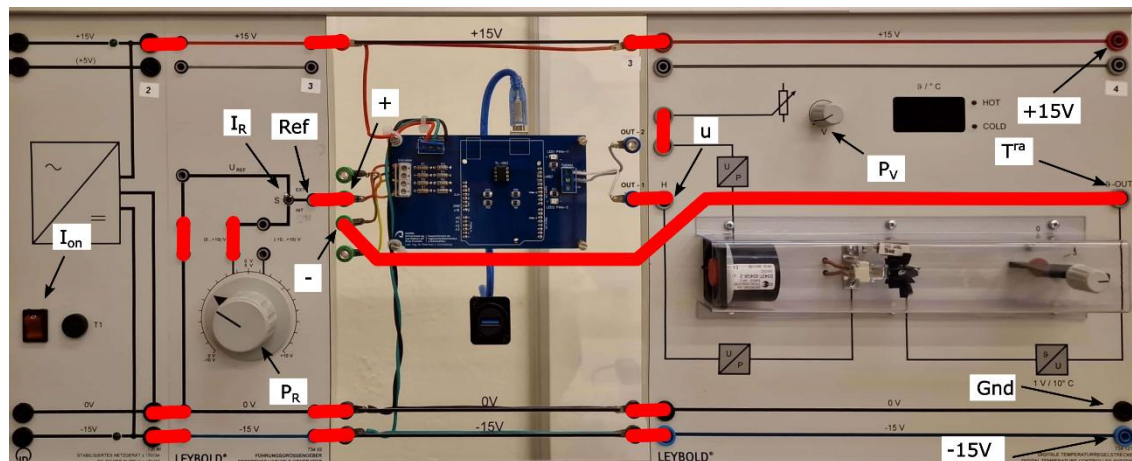


Figura 6.9.- Conexiones con módulo Arduino con control manual del ventilador.

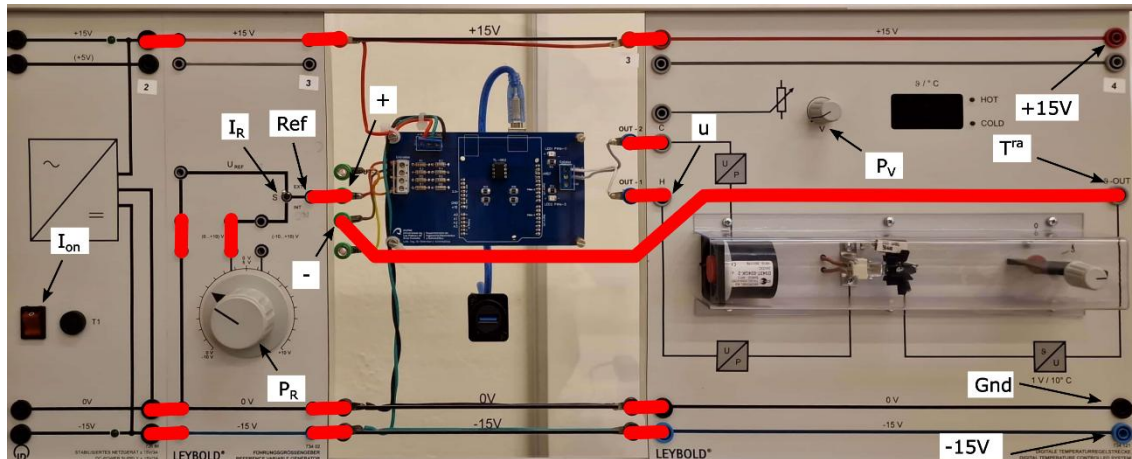


Figura 6.10.- Conexiones con módulo Arduino con control por Arduino del ventilador.

6.2.3.- Toma de datos (control con el IDE de Arduino)

Antes de comenzar la práctica, hay que tener en cuenta que la librería **TimerOne** no está instalada. Para instalarla hay que ir al apartado remarcado en la figura 6.2 **administrar bibliotecas**, poner buscar **Timerone** e **instalarla**.

Para la toma de datos se deben de seguir los siguientes pasos y si se utiliza el módulo Arduino se debe utilizar el cableado de la **figura 6.9**,

1. Asegurarse de que el interruptor $I_R = 0$ (interruptor hacia **arriba**).
2. Encender el sistema (interruptor $I_{on} = 1$).
3. Ajuste del potenciómetro del ventilador P_V según se muestra en la figura .
4. Poner la **válvula de mariposa** en posición **4 (horizontal)**.
5. El programa que se anexa al final del documento muestra el código de los controles a utilizar. **El programa sin comentar está diseñado para que la potencia de la lámpara se controle con el potenciómetro entrada Ref.** Cargar el programa en el Arduino.
6. **Activar** el interruptor I_R (interruptor **hacia abajo**).
7. Comprobar que se puede controlar la luminosidad de la lámpara variando el potenciómetro de referencia. Observar que el periodo de muestreo es de un segundo, es decir, que al variar el potenciómetro, la bombilla tarda un poco en responder.
8. **Activar** el **monitor serie** en el **IDE de Arduino** como se indica en la figura 6.2.
9. **Antes de desconectar ajustar la referencia**, la segunda columna que aparece en el monitor serie, a **40°C**.
10. **Desactivar** el interruptor I_R (interruptor **hacia arriba**).

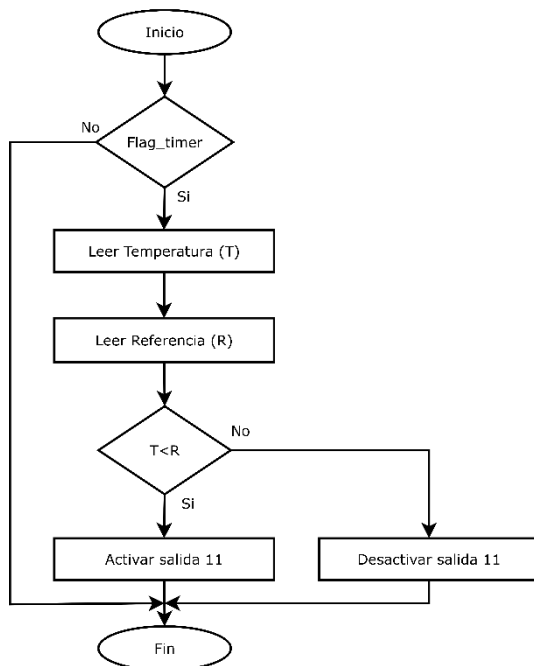


Figura 6.11.- Control todo-nada

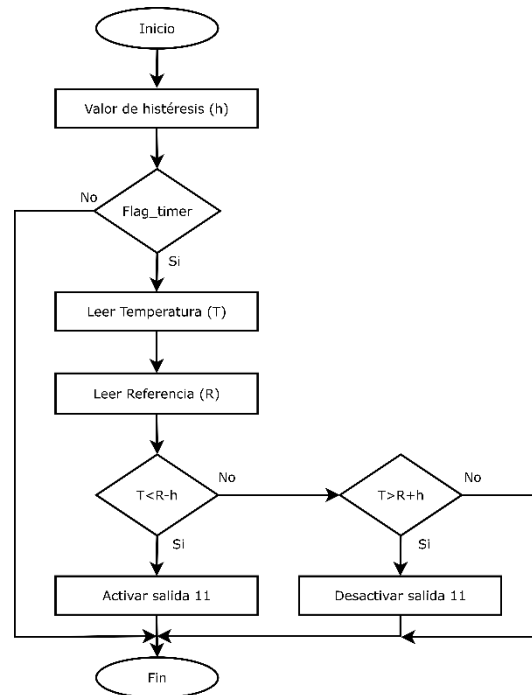


Figura 6.12: Control todo-nada con Histéresis

11. Poner el potenciómetro del ventilador al máximo.
12. **Descomentar** en el **programa** que se encuentra en el **anexo 6.1** la parte del **control todo-nada**. Para entender su funcionamiento se muestra el **diagrama de flujo** en la figura 6.11.
13. Si la temperatura del sistema es **inferior a 30°C**. **Ajustar el potenciómetro** del ventilador P_V según se muestra en la figura 
14. **Datos para el comienzo de la prueba**
 $V_{CR} = 4V$ (temperatura de referencia 40°C),
 Válvula Mariposa=4,
 Interruptor I_R **desactivado** (interruptor hacia arriba).
15. **En $t = 0$ segundos (t_0). (al unísono)**
activar el interruptor I_R (interruptor hacia abajo).
Limpiar la salida del monitor serie y anotar el valor que marca el tiempo (t_0).
16. Esperar **120 segundos**
17. **En $t = t_0 + 120$ segundos (Hay que sumarle 120 al primer valor del tiempo).**
 Poner la **válvula de mariposa** en posición **0 (vertical)**.
18. Esperar **120 segundos**
19. **En $t = t_0 + 240$ segundos (Hay que sumarle 240 al primer valor del tiempo).**
20. **Desactivar** el interruptor I_R (interruptor hacia arriba).
21. Poner el potenciómetro del **ventilador al máximo**.
22. Fin de toma de datos.

23. Deseleccionar **autoscroll** en el **monitor serie**.
24. Copiar el contenido del monitor serie en un fichero de texto y llamar al fichero **Control_TN.txt**.
25. Volver a seleccionar **autoscroll** en el **monitor serie**.
26. **Descomentar** el código para un **control todo-nada con una histéresis de 0,5°C**. El diagrama de flujo del programa se muestra en el diagrama de la figura 6.12.
27. **Repetir** los pasos del 13 al 25 y llamar al fichero **Control_TN_H.txt**.
28. **Descomentar** el código para un **control proporcional de ganancia K=20** cuyo diagrama de flujo se muestra en la figura 6.13.

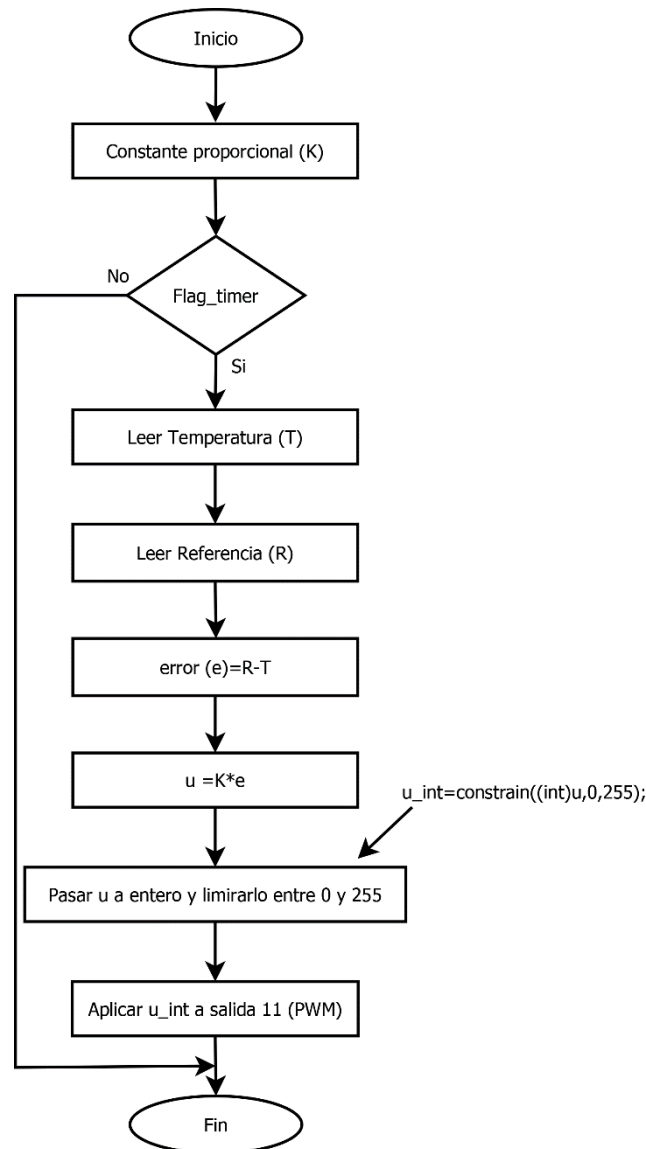


Figura 6.13.- Control proporcional.

29. **Repetir** los pasos del 13 al 25 y llamar al fichero **Control_P.txt**.
30. Fin de la práctica

6.2.4.- Toma de datos (control con Simulink)

Para la toma de datos se deben de seguir los siguientes pasos y utilizar las **conexiones** de la **figura 6.10**:

1. Encender el sistema (interruptor $I_{on} = 1$).
2. **Desconectar** la **conexión** marcada como **u** en la **figura 6.10** (esto evitará que se encienda la lámpara hasta que el modelo del Simulink esté ejecutándose).
3. Poner la **válvula de mariposa** en posición **4 (horizontal)**.
4. Ajustar con P_R un valor aproximado de **4V**.
5. **Activar** el interruptor I_R (interruptor **hacia abajo**)
6. Cargar en el PC el modelo Simulink **PID_Simulink_con_ventilador_R2021b.slx**. Dicho modelo se muestra en la **figura 6.14** y permite:
 - Controlar la **velocidad** del **ventilador** con la perilla de la **parte verde** o pinchando dos veces sobre el valor e **introduciéndolo manualmente** y de la **salida analógica 5** (modulada en ancho de pulso PWM).
 - Utilizar un **controlador PID** con el controlador programado en la **parte naranja** que permite ajustar los parámetros k , T_i y T_d .
 - Para utilizar el módulo de Simulink hay que definir el periodo de muestreo T a 1s. Para ello hay que ejecutar en el **Command Window de Matlab** el comando:
$$>> T = 1$$
 - Este programa de Simulink toma el valor de las entradas del Arduino **A0 (temperatura de consigna)** y **A1 (temperatura de la lámpara)** les aplica el controlador PID y genera la **acción de control** a través de la **salida analógica 11** (modulada en ancho de pulso PWM).
 - Este programa además genera un fichero llamado **t_R_T_AC_Aclim_Vent.mat** en el que se almacenan cada segundo el **tiempo**, la señal de **referencia** y la **temperatura** en grados centígrado, la **acción de control sin limitar y limitada** de 0 a 10V y la salida del **ventilador** en una señal de 0-100%. Este fichero es el que se deberá entregar y el utilizado para graficar las señales.
 - También permite en tiempo real ver el valor de las variables, errores, etc.

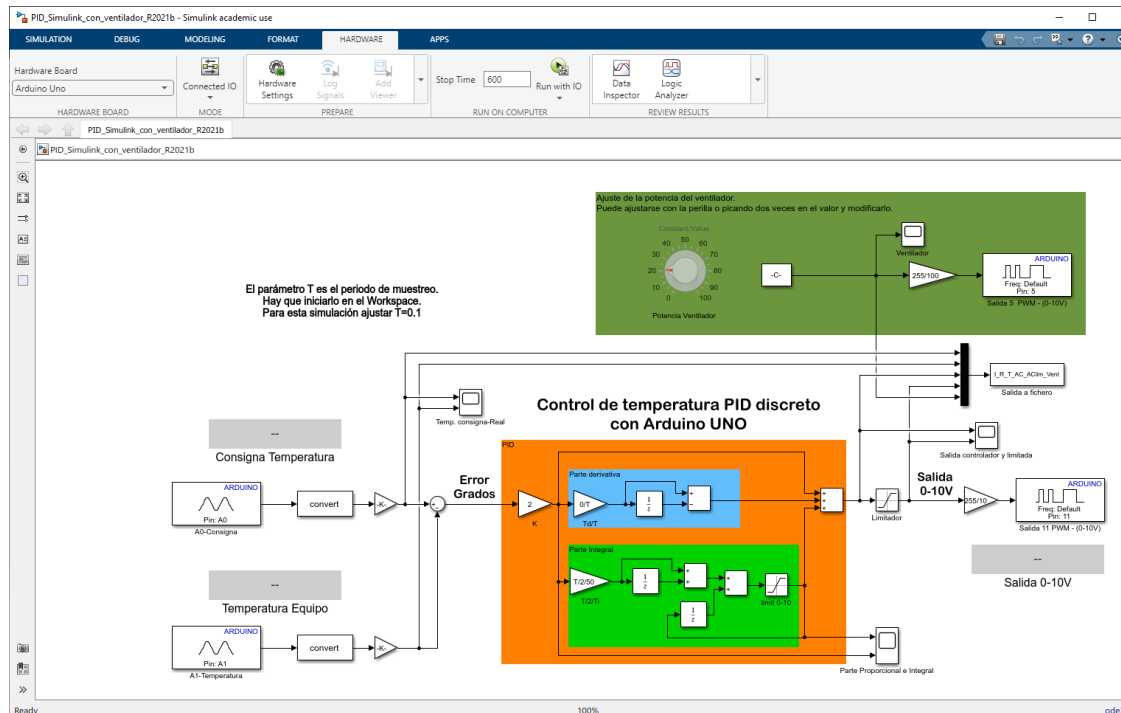


Figura 6.14.- Modelo para control PID con Simulink

7. **Ejecutar el modelo en Simulink.** Para ello seleccionar la pestaña **Hardware**, poner el **Stop Time** a **1500** y picar en el icono **Run with I/O**.
8. Una vez el modelo en ejecución **conectar la conexión u** de la **figura 6.10**.
9. **Ajustar** la potencia del **ventilador** al **10%**.
10. **Ajustar** el potenciómetro P_R para obtener en el modelo un valor en **Consigna Temperatura de 30°C (con una aproximación de $\pm 0,2^\circ\text{C}$)**.
11. Esperar **120 segundos** (Para todas las esperas: puede a ver un error de $\pm 2s$, ya que todas las variables quedarán grabadas en un fichero y se sabrá el momento exacto).
12. **Ajustar** el potenciómetro P_R para obtener en el modelo un valor en **Consigna Temperatura de 40°C (con una aproximación de $\pm 0,2^\circ\text{C}$)**.
13. Esperar **120 segundos**
14. **Ajustar** el potenciómetro P_R para obtener en el modelo un valor en **Consigna Temperatura de 30°C (con una aproximación de $\pm 0,2^\circ\text{C}$)**.
15. Esperar **120 segundos**
16. **Ajustar** la potencia del **ventilador** al **100%**.
17. Esperar **120 segundos**
18. **Ajustar** la potencia del **ventilador** al **10%**.
19. Esperar **120 segundos**
20. Según **elección** del estudiante **cambiar** el ajuste de **k** al **doble** o a la **mitad y/o** el de T_i también al **doble** o a la **mitad**.
21. **Ajustar** el potenciómetro P_R para obtener en el modelo un valor en **Consigna Temperatura de 40°C (con una aproximación de $\pm 0,2^\circ\text{C}$)**.
22. Esperar **120 segundos**

23. **Ajustar** el potenciómetro P_R para obtener en el modelo un valor en **Consigna Temperatura de 30°C (con una aproximación de $\pm 0,2^\circ\text{C}$)**.
24. Esperar **120 segundos**.
25. Según **elección** del estudiante **cambiar** el ajuste de **k** al **doble** o a la **mitad** y/o el de T_i también al **doble** o a la **mitad**, o a el **valor** que se **desea** en ambos parámetros.
26. **Ajustar** el potenciómetro P_R para obtener en el modelo un valor en **Consigna Temperatura de 40°C (con una aproximación de $\pm 0,2^\circ\text{C}$)**.
27. Esperar **120 segundos**.
28. **Picar** en el icono de **stop** en el **Simulink**
29. Salvar el fichero **t_R_T_AC_Aclim_Vent.mat** que se encuentra en el **Current Folder** en la **parte superior izquierda** del **Command Windows** de **Matlab**.
30. Fin de la práctica

6.3 En casa

Una vez realizada la práctica, se tendrán los datos de los dos experimentos, las relacionadas con un control con el Arduino programado con el IDE de Arduino y las realizadas con el Simulink.

6.3.1 Tareas relacionadas con el IDE de Arduino

Una vez realizada la toma de datos, los estudiantes dispondrán de **tres ficheros** con la **respuesta** correspondiente a los distintos controles.

Partiendo de cada fichero obtenido en el apartado de la toma de datos, cada estudiante deberá crear una variable en Matlab llamada **t_incT_T_v** con las siguientes columnas (la variable es similar a la creada en la práctica 1),

- Columna 1: Tiempo en s
- Columna 2: Incremento de temperatura respecto a la temperatura ambiente en °C.
- Columna 3: Temperatura en °C.
- Columna 4: La tensión aplicada al equipo. Este valor se añadirá manualmente y debe ser constante en toda la variable.

6.3.1.1.- Salvamos la variable **t_incT_T_v** de cada prueba en un fichero de Matlab llamado **Control_TN.mat**, **Control_TN_H.mat** y **Control_P.mat** según corresponda.

6.3.1.2.- Representar gráficamente la **temperatura del sistema** respecto al **tiempo** de cada prueba.

6.3.1.3.- Rellenar la siguiente tabla para cada prueba

Prueba	Tiempo establecimiento ($\pm 1^\circ\text{C}$)	Sobreoscilación (%)	Temperatura final
Control_TN			
Control_TN_H			
Control_P			

6.3.1.4.- En vista de la tabla anterior y de las gráficas ¿Los ajustes de qué prueba elegirías para el control del sistema? Razona tu respuesta

6.3.1.5.- En vista a los resultados del control todo nada con y sin histéresis. Cual ha dado unos resultados mejores. Explica en que casos es conveniente utilizar un control todo nada con histéresis, aunque los resultados sean peores.

6.3.2 Tareas relacionadas con el Simulink

Una vez realizada la toma de datos, los estudiantes dispondrán de un fichero con la respuesta correspondiente al control con el PID propuesto.

6.3.2.1.- Representar gráficamente respecto al **tiempo** los datos de la prueba. Es decir, la **temperatura de consigna** y la del **sistema**, la **acción de control sin limitar y limitada** de 0 a 10V y la salida del **ventilador** en una señal de 0-100%.

En este fichero, se observan claramente los cambios de consigna y los momentos en los que se cambia la velocidad del ventilador.

6.3.2.2.- ¿Cuándo se limita más la acción de control por saturación de la acción integral?, ¿Ante cambios de consigna o ante la perturbación del ventilador?

6.3.2.3.- A la vista de las gráficas y de las respuestas del control PID ante el cambio de consigna de 30°C a 40°C. ¿Qué juego de parámetros **K** y **T_i** elegirías de los que has utilizado? **Razona tu respuesta.**

Anexo 6.1:

Programa de Arduino de ejemplo para ver cómo utilizar los comandos.

Programa para el Arduino Uno y con los controles utilizados en esta práctica.

```
/*
Autor: José J. Quintana
Fecha creación: 10/05/2021
Fichero: PWM_Prodel_AI_int_v4
Universidad de Las Palmas de Gran Canaria

Programa base para la utilización del arduino uno como elemento de control, en
la asignatura de Regulación Automática del grado de Ingeniería en Electrónica
Industrial y Automática.
Utiliza una interrupción para un periodo de muestreo ajustable (se ha ajustado a 1s.
Utiliza las entradas analógicas y salidas PWM del arduino.

Implementa:
Un control manual en que La salida PWM (lámpara) sigue al potenciómetro de consigna
del sistema térmico (entrada A1).
Un control todo-nada.
Un control todo-nada con histéresis.
Un controlador P, PD, PI y PID.
*/

// Se incluyen las librerías.
#include "TimerOne.h" // Se necesita para utilizar la interrupción de tiempo

// Se definen las entradas y salidas
const int Pin_PWM_11 = 11;

const int Pin_Ref_A0 = A0; // Entrada de la referencia de temperatura
const int Pin_Tra_A1 = A1; // Entrada del sensor de temperatura

long tiempo; // variable para almacenar el tiempo

int valor_tra, valor_ref, salida_manual, u_int;
float temperatura, referencia;
float error, u_float, yp, yd, yI, h=0.5;
float k=20, Td=2, Ti=50, T=1;
float errorI=0, yI1=0;

boolean flag_timer=false;

void setup() {
    // Se inicializa la interrupción de tiempo
    Timer1.initialize(1000000); // set a timer of length 1000000 microseconds
    // (or 1 sec - or 1Hz)
    Timer1.attachInterrupt( timerIsr ); // attach the service routine here

    // Se activa la referencia analógica externa
    // ;;; Conectar en placa AREF con 3.3V !!!
    analogReference(EXTERNAL);

    // Se define la salida 11 como salida
    pinMode(Pin_PWM_11, OUTPUT);
    // initialize serial communications at 9600 bps:
    Serial.begin(9600);
}

void loop() {
    if (flag_timer) // Este if se activa cada segundo por la función timerIsr
    {
        tiempo=tiempo + 1; // Se genera el valor del tiempo en segundos

        // Se guardan los valores anteriores del error y de la parte integral
        errorI=error;
        yI1=yI;

        // Se lee el valor de las entradas analógicas como valores enteros.
        valor_tra = analogRead(Pin_Tra_A1);
        valor_ref = analogRead(Pin_Ref_A0);
    }
}
```

```
// Se escalan las entradas 0-1023 ud (entero)= 0-100 grados centigrados (float)
temperatura=(float)valor_tra*100/1023; // El comando (float) pasa el entero valor_tra a float
referencia=(float)valor_ref*100/1023;

////////// Controles propuestos //////////
// Control 1
// Control manual utilizando la entrada A0 (Potenciómetro).

u_int=map(valor_ref,0,1203,0,255); // La salida PWM sólo acepta valores entre 0 y 255
analogWrite(Pin_PWM_11, u_int);

// // Control 2
// // Control todo nada sin histéresis
// if (temperatura < referencia)
// {
//   u_int=255; // salida al 100%
//   analogWrite(Pin_PWM_11, u_int);
// }
// else
// {
//   u_int=0; // salida al 0%
//   analogWrite(Pin_PWM_11, u_int);
// }

// // Control 3
// // Control todo nada con histéresis
// if (temperatura < (referencia-h))
// {
//   u_int=255; // salida al 100%
//   analogWrite(Pin_PWM_11, u_int);
// }
// else if (temperatura > (referencia+h))
// {
//   u_int=0; // salida al 0%
//   analogWrite(Pin_PWM_11, u_int);
// }

// // Control 4
// Controlador P, PI, PD y PID

// Se calcula el error
error=referencia-temperatura;

// Control proporcional
yp=k*error; // Se calcula la acción proporcional

// Control derivativo
yd=k*Td/T*(error-error1);

// Acción integral
yI=yI1+k/Ti*T/2*(error+error1);
// La acción integral se limita para que no haya saturación (antiwindup)
if (yI>255) {yI=255;}
else if (yI<0) {yI=0;}

// // Control P
// u_float=yp;

// // Control PD
// u_float=yp+yd;

// // Control PI
// u_float=yp+yI;

// PID
// u_float=yp+yd+yI;

// // Se envía a la salida PWM
// u_int=(int)u_float; // se pasa a un valor entero
// u_int=constrain(u_int,0,255); // se limita a un valor entre 0 y 255
// analogWrite(Pin_PWM_11, u_int); // se envía a la salida PWM

//Serial.print("tiempo = "); // En caso de que se quiera mostrar por el puerto serie
Serial.print(tiempo); //tiempo en décimas de segundo
Serial.print("\t "); // Tabulación
//Serial.print("\t referencia = ");
Serial.print(referencia);
Serial.print("\t ");
//Serial.print("\t temperatura = ");
Serial.print(temperatura);
Serial.print("\t ");
```

```
//Serial.print("\t salida = ");  
Serial.print(u_int);  
Serial.print("\n"); // Nueva línea  
  
    flag_timer=false;  
}  
}  
  
/// -----  
/// Custom ISR Timer Routine  
/// -----  
// Esta función se activa cada vez que pasa el tiempo definido en la interrupción.  
// En nuestro caso cada segundo  
void timerIsr()  
{  
    flag_timer=true;  
}
```